تمرین کلاس — serve کردن model به صورت local

تو این تمرین اول یک chat model رو روی لپتاپ serve می‌کنیم. بعد همون فایل رو با یک runtime دیگه بالا میاریم. آخرش یک decision model مسیر رو انتخاب می‌کنه و MCP یادداشت رو از داخل script جدا می‌کنه و می‌ده به pipeline.

CPU. فقط این modelها: `qwen2.5:0.5b` و `tev1:0.8b` . Ollama نسخه 0.35 یا جدیدتر:

`ollama ––version`

---

۱. Ollama را serve کنید

الان به‌جای API ابری، خود Ollama رو روی لپتاپ serve می‌کنیم و chat model یعنی `qwen2.5:0.5b` رو local بالا میاریم. اولین جواب رو از CLI می‌گیریم که مطمئن شیم هم server بالاست، هم model آماده است.

```
ollama serve
ollama pull qwen2.5:0.5b
ollama list
ollama show qwen2.5:0.5b
ollama run qwen2.5:0.5b "Say hello in one sentence."
ollama ps
```

`ollama list` مدل‌های download شده را نشان می‌دهد. `ollama show` مشخصات `qwen2.5:0.5b` را نشان می‌دهد. `ollama ps` می‌گوید این model همین الان در حافظه هست یا نه، و روی CPU است یا GPU.

اگر port `11434` از قبل بالا بود، `ollama serve` رو دوباره نزنید.

```
curl http://127.0.0.1:11434/api/tags
```

خروجی `ollama run` رو ببینید. model باید local جواب داده باشه.

---

۲. همان model را با HTTP صدا بزنید

server بخش ۱ هنوز روشنه. دیگه توی terminal چت نمی‌کنیم. از HTTP به همون server درخواست می‌زنیم و role و temperature و max token رو عوض می‌کنیم.

system message و user message ثابت:

```
system: You are a concise teaching assistant. Answer in one English sentence.
user:   What is quantization?
```

native API

اینجا درخواست مستقیم به API خود Ollama می‌ره. تنظیم‌های generation رو می‌ذاریم زیر `options` .

دو بار `POST http://127.0.0.1:11434/api/chat` با `stream: false` . `temperature` و `num_predict` زیر `options` هستن.

- بار اول: `temperature` = `0` ، `num_predict` = `40`
- بار دوم: `temperature` = `0.8` ، `num_predict` = `40`

هر دو response رو print کنید.

OpenAI-compatible API

همون server و همون model مونده. این بار شکل request رو شبیه OpenAI می‌فرستیم، برای همین اسم فیلد max token عوض می‌شه.

یک بار `POST http://127.0.0.1:11434/v1/chat/completions` . اینجا max token فیلد `max_tokens` است و مقدارش `40` . `temperature` = `0` . `requests` کافیه.

---

۳. llama.cpp را local serve کنید

Ollama رو خاموش نمی‌کنیم. همون فایل model رو این بار با `llama-server` و `-ngl 0` روی CPU سرو می‌کنیم تا یک server دوم روی port `8082` جواب بده. `llama-cli` همون فایل رو بدون HTTP، مستقیم اجرا می‌کنه.

GGUF رو دوباره download نکنید. digest رو از اینجا بردارید:

```
ollama show qwen2.5:0.5b --modelfile
```

فایل blob:

```
macOS / Linux:  ~/.ollama/models/blobs/sha256-<digest>
Windows:        %USERPROFILE%\.ollama\models\blobs\sha256-<digest>
```

اگر `llama-cli` این فایل رو باز نکرد، GGUF کارگاه رو بگذارید:

`qwen2.5-0.5b-instruct-q4_k_m.gguf` . binary آماده llama.cpp رو نصب کنید. از source هم

build نکنید. روی بعضی سیستم‌ها اسم فایل `llama-cli.exe` و `llama-server.exe` است.

terminal رو باز نگه دارید:

```
llama-server -m <GGUF> -c 2048 -ngl 0 --host 127.0.0.1 --port 8082 --alias
classroom-gguf
```

`-ngl 0` یعنی CPU. این flag رو برندارید.

```
curl http://127.0.0.1:8082/health
curl http://127.0.0.1:8082/v1/models
```

بعد همون user message رو به `POST http://127.0.0.1:8082/v1/chat/completions` بفرستید. `model` همون id است که `/v1/models` داده. `max_tokens` = `40`.

یک بار هم با CLI:

```
llama-cli -m <GGUF> -p "What is quantization?" -n 40 -c 2048 -ngl 0
```

---

۴. decision model

تا اینجا chat model جواب رو به صورت متن می‌نوشت. الان `tev1:0.8b` رو روی همون Ollama بالا میاریم. این model جمله نمی‌سازه و فقط `choice` برمی‌گردونه.

```
ollama pull tev1:0.8b
```

خروجی این model فیلد `choice` و `confidence` است. endpoint آن `/v1/systemone` است. با `llama-cli` و `/v1/chat/completions` صداش نزنید.

توی CLI، مقدار state رو از stdin بدین:

```
ollama run tev1:0.8b \
  --ask "Which route should answer this user message?" \
  --choice lookup="The answer should come from the stored bootcamp notes." \
  --choice direct="The user asks a general question and does not need the stored
notes." \
  --format json < question.txt
```

همون decision رو در Python هم بگیرید:

```
POST http://127.0.0.1:11434/v1/systemone

{
  "model": "tev1:0.8b",
  "state": "<user message>",
  "questions": {
    "route": {
      "type": "choice",
      "instructions": "Which route should answer this user message?",
      "criteria": {
        "lookup": "The answer should come from the stored bootcamp notes.",
        "direct": "The user asks a general question and does not need the stored
notes."
      }
    }
  }
}
```

JSON هر دو call رو print کنید. `choice` رو دستی عوض نکنید.

---

۵. MCP و pipeline

یادداشت رو دیگه داخل script به صورت hardcode نمی‌ذاریم. یک MCP server جداش می‌کنه و به صورت tool و resource و prompt درمیاره. decision model مسیر رو انتخاب می‌کنه و chat model فقط sentence آخر رو می‌نویسه.

`bootcamp_server.py` رو کپی نکنید. توی `study_server.py` این data و سه تا primitive رو ثبت کنید:

```
NOTES = {
    "16": "Week 16 covers MCP: tools, resources, prompts, discovery, and a bounded
controller.",
    "17": "Week 17 covers local inference with Ollama and llama.cpp. CPU runs use
-ngl 0.",
}
```

- tool: `find_note(topic: str) -> dict`

`topic` رو lowercase کنید. اگر `topic` داخل id یا text بود، اون note بیاد توی نتیجه. خروجی: `topic`، لیست matchها، `match_count`. اگر match نبود error ندید و `match_count` رو `0` بگذارید.

- resource: `note://{note_id}`

text همون note رو برگردونه. اگر id ناشناخته بود، یک جمله not-found برگردونید، بدون exception.

- prompt: `answer_from_note(note: str)`

یک system instruction انگلیسی برگردونه. اگر `note` پر بود، بگه فقط از همون note و در یک sentence جواب بده. اگر `note` خالی بود، بگه هیچ noteای در اختیارش نیست و در یک sentence جواب بده.

توی `assistant.py` با `async with Client(...)` اینا رو print کنید:

- `list_tools`
- `list_resource_templates`
- `list_prompts`
- `read_resource("note://16")`
- `get_prompt("answer_from_note", ...)` یک بار با note هفته ۱۶، یک بار با empty string

توی controller فقط `find_note` توی allow-list باشد.

این دو تا user message رو جدا جدا بفرستید توی pipeline:

```
A) Which week covers MCP tools, resources, and prompts?
B) What does temperature do in text generation? Answer in one sentence.
```

برای هر کدام:

1. از `/v1/systemone` مقدار `choice` و `confidence` رو بخونید.
2. policy: اگر `choice == "lookup"` یا `confidence < 0.55`، route می‌شه `lookup`. وگرنه `direct`.
3. اگر route برابر `lookup` بود، فقط `find_note` رو صدا بزنید. text همان matchها رو بدین به prompt. اگر match خالی بود، `note` خالی بمونه.
4. اگر route برابر `direct` بود، tool call نکنید و `note` رو خالی بگذارید.
5. system message مدل chat همون textی باشه که prompt برگردونده.
6. جواب رو از `qwen2.5:0.5b` بگیرید. `temperature = 0`، حداکثر ۶۰ token.

برای هر message اینا رو چاپ کنید: `choice`، `confidence`، route بعد از policy، نتیجه `find_note` اگر صدا زده شد، system message، و sentence مدل chat.

اگر serve بالا نیامد

اگر یک مرحله بالا نیومد، قبل از ادامه اینا رو چک کنید.

- `connection refused` روی `11434` : اپ Ollama رو باز کنید، بعد `GET /api/tags` .
- `model not found` : tag رو از `ollama list` بردارید (`:0.5b` و `:0.8b`).
- `/v1/systemone` خطا داد: Ollama قدیمی‌تر از 0.35 است.
- `llama-server` روی loading موند: `GET /health` رو بعد از تموم شدن log دوباره بزنید. terminal سرور رو نبندید.